

**INSTITUTO
FEDERAL**

Goiano

Campus
Trindade

Sistemas Distribuídos

Invocação de Método Remoto (RMI)

Java · RMI Registry · Stubs

Prof. Dr. Renato de Sousa Gomide

Objetivos da aula

- Entender **por que** chamadas remotas existem: **RPC** como conector de sistemas distribuídos
- Compreender **stub**, **marshalling** e **transparência de acesso**
- Conhecer a arquitetura do **Java RMI** e o papel do **RMI Registry**
- Implementar um serviço remoto: **interface**, **implementação**, **servidor** e **cliente**
- Reconhecer as **limitações** de RMI e compará-lo com **gRPC** e **REST**

Onde isso se encaixa

Em uma arquitetura distribuída, **componentes** trocam informação por meio de **conectores**.

Conector é o mecanismo que **media** a comunicação, coordenação ou cooperação entre componentes.

Exemplos: **chamada de procedimento remoto (RPC)**, **passagem de mensagens**, **streaming**.

Nesta aula o conector é a **chamada remota síncrona**: o cliente **bloqueia** até a resposta chegar.

O problema: código que mora em outra máquina

Chamar um método local é trivial:

```
String r = obj.sayHello();
```

Chamar o **mesmo método** em um processo que roda em **outro computador** exige:

1. Abrir uma **conexão de rede**
2. **Serializar** nome do método e argumentos
3. Enviar bytes, esperar, **desserializar** a resposta
4. Tratar falhas de rede que **não existem** localmente

RPC/RMI escondem os passos 1–3 e obrigam você a lidar com o passo 4.

RPC — *Remote Procedure Call*

- Um **componente** de uma aplicação manda uma requisição para **outro componente** usando a sintaxe de uma **chamada de procedimento local**
- Proposto por **Birrell e Nelson (1984)**
- Objetivo: **transparência de acesso** — o programador escreve o mesmo código para local e remoto

RMI (*Remote Method Invocation*) é a evolução **orientada a objetos** do RPC: em vez de procedimentos, invocamos **métodos de objetos remotos**.

Desvantagem fundamental

A **desvantagem** de RPC e RMI é que o **chamador** e o **chamado** precisam estar **ativos** e **em funcionamento** durante todo o tempo da comunicação.

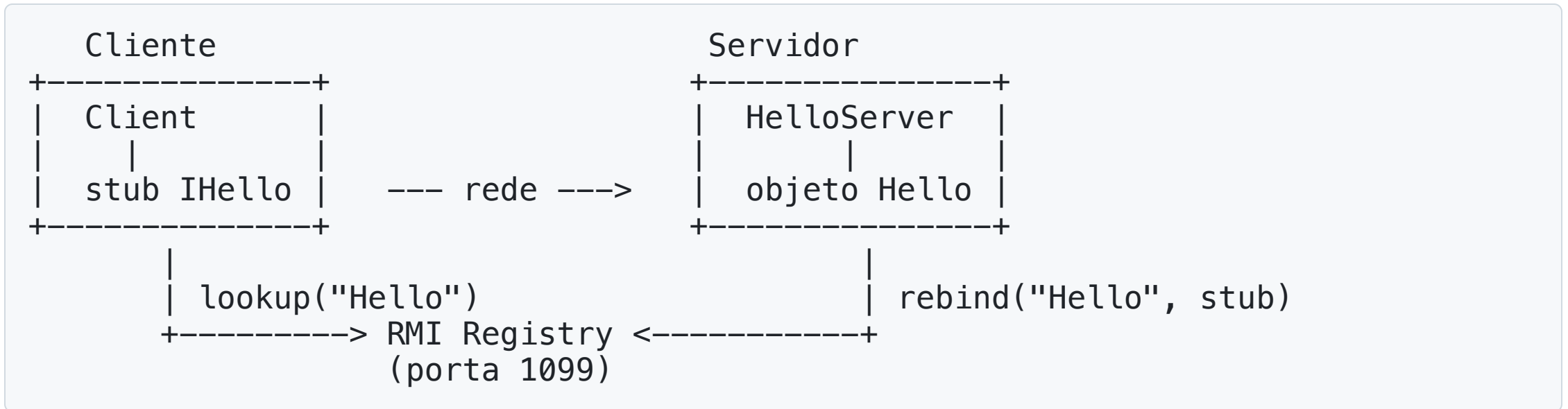
- Comunicação **síncrona** e **acoplada no tempo**
- Servidor fora do ar → cliente recebe **exceção**, não uma fila
- Alternativa desacoplada: **Middleware Orientado a Mensagens (MOM)** e **PubSub**

Stub e marshalling

Termo	Papel
Stub (lado cliente)	Objeto local que finge ser o objeto remoto; empacota a chamada
Skeleton (lado servidor)	Desempacota a chamada e a repassa ao objeto real (implícito no Java moderno)
Marshalling	Converter argumentos em bytes para trafegar na rede
Unmarshalling	Reconstruir os objetos do outro lado

Analogia: o **stub** é um **despachante**. Você fala com ele como se fosse o órgão público; ele resolve papelada, protocolo e transporte.

Arquitetura do Java RMI



O **Registry** é um **serviço de nomes**: mapeia um **nome** ("Hello") para um **stub** de objeto remoto.

Passo a passo de um serviço RMI

1. Definir a **interface remota** — o contrato (`extends Remote`)
2. **Implementar** a interface no servidor
3. **Exportar** o objeto (gera o stub) e **registrar** com um nome
4. Cliente faz **lookup** no Registry e recebe o **stub**
5. Cliente invoca métodos **no stub** como se fossem locais

A **interface** é o único código que **cliente e servidor compartilham** — por isso ela fica no pacote `common` .

1. A interface remota

src/main/java/common/hello/IHello.java

```
package common.hello;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface IHello extends Remote {
    String sayHello() throws RemoteException;
}
```

- **extends Remote** — marca a interface como invocável remotamente
- **throws RemoteException** — obrigatório em **todo** método remoto: a rede pode falhar

Por que `RemoteException` é obrigatória?

Uma chamada local só falha por **bug**. Uma chamada remota falha também por:

- Servidor **fora do ar** ou reiniciado
- **Timeout** ou perda de pacotes
- Objeto **não registrado** no Registry
- Erro de **serialização** de argumento

O compilador **força** você a admitir que remoto $\neq$ local. A transparência de acesso é **parcial** — e isso é proposital.

2. A implementação

```
src/main/java/server/hello/Hello.java
```

```
package server.hello;

import common.hello.IHello;

public class Hello implements IHello {

    @Override
    public String sayHello() {
        System.out.println("Hello.sayHello() called");
        return "Hello World!";
    }
}
```

A implementação vive **só no servidor**. O cliente nunca vê esta classe — só a interface.

3. 0 servidor

src/main/java/server/hello/HelloServer.java

```
Hello obj = new Hello();
IHello skeleton = (IHello) UnicastRemoteObject.exportObject(obj, 0);
Registry registry = LocateRegistry.createRegistry(1099);

// SE ESTIVER EXECUTANDO 0 rmiregistry
// Registry registry = LocateRegistry.getRegistry();

registry.rebind("Hello", skeleton);
System.out.println("Server is ready!");
```

Dissecando o servidor

Linha	O que faz
<code>exportObject(obj, 0)</code>	Torna o objeto disponível para chamadas remotas e devolve o stub . O <code>0</code> = porta anônima escolhida pela JVM
<code>createRegistry(1099)</code>	Sobe um Registry dentro do próprio processo — dispensa o <code>rmiregistry</code> externo
<code>getRegistry()</code>	Alternativa: conecta a um <code>rmiregistry</code> já em execução
<code>rebind("Hello", skeleton)</code>	Publica o skeleton sob o nome <code>"Hello"</code> , substituindo um registro anterior

`bind` falha se o nome já existir; `rebind` sobrescreve.

4. O cliente

src/main/java/client/hello/Client.java

```
String host = (args.length < 1) ? null : args[0];
try {
    Registry registry = LocateRegistry.getRegistry(host);
    IHello stub = (IHello) registry.lookup("Hello");
    String response = stub.sayHello();
    System.out.println("response: " + response);
} catch (Exception e) {
    System.err.println("Client exception: " + e.toString());
}
```

host == null → **localhost**. Passe o IP como argumento para acessar outra máquina.

O que acontece em `stub.sayHello()`

1. O **stub** serializa a invocação (identificador do objeto + método + argumentos)
2. Abre/reusa uma conexão **TCP** com a porta exportada do servidor
3. O runtime RMI do servidor **desserializa** e chama `Hello.sayHello()`
4. O retorno volta serializado; o stub o **desserializa** e devolve
5. O cliente vê apenas: `String response = "Hello World!"`

O `System.out.println` de `Hello.sayHello()` aparece no **terminal do servidor** — prova de que o código executou lá.

Passagem de parâmetros

Tipo do parâmetro	Como trafega
Primitivos e objetos <code>Serializable</code>	Por valor — uma cópia vai pela rede
Objetos que implementam <code>Remote</code>	Por referência remota — vai o stub
Objeto não serializável	Erro em tempo de execução

Consequência prática: alterar um objeto recebido por valor **não** afeta o original do outro lado.

Executando o projeto

Compilar:

```
mvn compile
```

Servidor (a partir da raiz do projeto):

```
java -classpath ./target/classes \  
-Djava.rmi.server.codebase=file:target/classes/ \  
server.hello.HelloServer
```

Cliente (em **outro terminal**):

```
java -classpath ./target/classes client.hello.Client
```

rmiregistry e codebase

- Usando `LocateRegistry.getRegistry()` é preciso iniciar o utilitário `rmiregistry` (em `$JAVA_HOME/bin`) a partir da pasta `target/classes`
- Com `createRegistry(1099)` o Registry sobe embutido — **não** precisa do utilitário
- `java.rmi.server.codebase` informa **de onde** baixar as classes que o outro lado ainda não tem

Ordem importa: suba o **servidor** antes do **cliente**, senão o `lookup` lança `NotBoundException`.

Erros comuns

Sintoma	Causa provável
<code>ConnectException: Connection refused</code>	Servidor ou <code>rmiregistry</code> não está no ar
<code>NotBoundException: Hello</code>	Nome não registrado (cliente subiu antes do servidor)
<code>ClassNotFoundException</code>	<code>codebase</code> /classpath sem a interface compartilhada
<code>NoSuchObjectException</code>	Objeto foi coletado ou não exportado
Trava em rede remota	Firewall : RMI usa 1099 e uma porta dinâmica

Limitações do RMI

- **Só Java** — cliente e servidor na mesma plataforma
- **Acoplamento**: mudar a interface exige **recompilar os dois lados**
- **Portas dinâmicas** dificultam firewall e NAT
- **Síncrono e acoplado no tempo** — ambos precisam estar ativos
- **Serialização Java** já foi vetor de **vulnerabilidades** conhecidas

Por isso, sistemas heterogêneos modernos preferem **gRPC** (contrato em `.proto`, multiplataforma) ou **REST**.

RMI × gRPC × REST

	Java RMI	gRPC	REST
Contrato	Interface Java	Arquivo <code>.proto</code>	Documentação/OpenAPI
Linguagens	Só Java	Multiplataforma	Multiplataforma
Transporte	JRMP sobre TCP	HTTP/2	HTTP/1.1
Formato	Serialização Java	Protobuf (binário)	JSON (texto)
Descoberta	RMI Registry	DNS / service mesh	URL
Streaming	Não	Sim (bidirecional)	Limitado

Resumo

- **RPC/RMI** são **conectores** que dão **transparência de acesso** a chamadas entre processos
- O **stub** empacota a chamada (**marshalling**) e a envia pela rede
- Java RMI: **Remote** + **RemoteException** + **exportObject** + **Registry**
- O **Registry** é um serviço de **nomes**: **rebind** publica, **lookup** recupera
- Chamador e chamado precisam estar **ativos ao mesmo tempo** — acoplamento temporal
- Parâmetros **Serializable** vão **por valor**; objetos **Remote** vão **por referência**

Referências

- [Getting Started Using Java RMI](#) — tutorial que originou este projeto
- [Java RMI — Trail do Java Tutorials](#)
- `java.rmi` — API